

AirSpy: An Air-pinch Keystroke Inference Attack on Virtual Keyboards in VR Systems

Luo Zhou, Jiusong Luo, Xueying Gong, Jiaming Pei, Wei Liu*, Huixiang Wen*

Abstract—Consumer VR headsets increasingly support text entry via air-pinch gestures on virtual keyboards, yet app-visible telemetry can inadvertently expose keystrokes. We present AirSpy, a permission-lean application-layer side channel that combines (i) VR hand-tracking joint pose with unit-quaternion rotations and (ii) lightweight per-frame PAL counters (e.g., draw calls and frame timing). Our key insight is that each keystroke induces short, repeatable PAL spikes/dips that align in time with subtle quaternion dynamics of the pinching hand. Based on this complementarity, we build a two-stage pipeline: a PAL-driven segmentation module anchors candidate keypress windows, followed by attribution-guided feature selection and a lightweight Transformer–DNN decoder for key inference. We implement AirSpy on Meta Quest 2 and Meta Quest 3 and evaluate it with 10 participants on a 29-key virtual keyboard (55k labeled frames). In foreground mode, AirSpy reaches 93.56% Top-1 accuracy with 92.8% segmentation recall; under background throttling, accuracy remains about 90.1%. Cross-device transfer between Quest 2 and Quest 3 maintains about 85–86% accuracy. The runtime overhead is small (about 11–13 ms added latency with under 3 FPS impact). Finally, we quantify practical mitigations, including API rate limiting and PAL counter sanitization, and summarize security–utility trade-offs to help platform designers better balance responsiveness and privacy.

Index Terms—Virtual reality; VR security; side channel attacks; keystroke inference; hand tracking; quaternions; runtime telemetry; Meta Quest.

I. INTRODUCTION

IMMERSIVE Virtual Reality (VR) and Augmented Reality (AR) platforms increasingly mediate sensitive user inputs—passwords, PIN codes, and private text—inside everyday applications and system dialogs. While vendors keep hardening traditional attack surfaces, the rich sensing and rendering pipelines of Extended Reality (XR) systems inadvertently create *side channels* that can reveal user intent without screen/audio capture or privileged access. Prior studies report leakage through runtime/engine statistics, motion traces, and even encrypted traffic patterns [1]–[4]; for instance, encrypted network traffic in AR can expose user activities [5], [6], and WebXR contexts can leak hand-motion signals sufficient for

This work was supported in part by the National Natural Science Foundation of China (Grant No. 62502010).

Luo Zhou, Jiusong Luo, and Xueying Gong are with the School of Computer Science and Technology, Donghua University, Shanghai 201620, China (e-mail: zhouluo@mail.dhu.edu.cn, jiusongluo@mail.dhu.edu.cn, willing499@163.com)

Huixiang Wen is with School of Medical Imaging, Bengbu Medical University, Bengbu 233030, China (e-mail: huixiangwen@bbmu.edu.cn)

Jiaming Pei is with the School of Computer Science, The University of Sydney, Sydney, 2006, NSW, Australia, (e-mail: jpei0906@uni.sydney.edu.au).

Wei Liu is with the School of Management Science and Engineering, Anhui University of Finance and Economics, Bengbu 233030, China (e-mail: liuwei628@aufe.edu.cn).

* corresponding author.

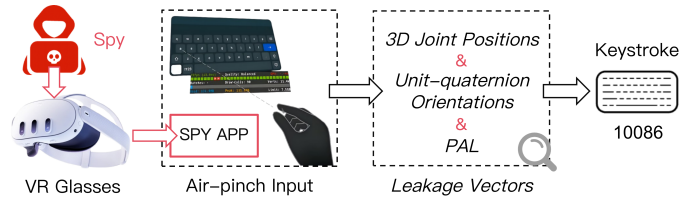


Fig. 1. Illustration of AirSpy for keystroke inference in VR systems.

passive activity and content tracking [7]. These observations motivate a systematic reassessment of input security on commodity headsets.

Existing XR keystroke-inference studies span several lines but share structural constraints. Single-modality approaches based on hand motion/air-typing [8], HMD/IMU signals [9], or avatar-skeleton kinematics [10] demonstrate feasibility, yet often under-represent the fine-grained *orientation* dynamics that govern natural air-pinch input (cf. Fig. 1) and may rely on stable sampling or controlled setups [11]. Camera-based attacks [12] and external-sensor variants (e.g., mmWave or acoustics) [13], [14] reduce deployability by requiring extra hardware or favorable vantage points, while network-/state-centric leakages (e.g., encrypted traffic, spatial traces, or shared-state services) [1], [2], [5], [15]–[19] do not directly yield robust *per-keypress* decoding under app-level constraints. Moreover, many evaluations assume foreground execution with high, stable sampling, and performance can degrade under background throttling and runtime/device drift; and signal attribution is often limited despite growing emphasis on explainable security analysis [20]–[24]. These gaps motivate a permission-realistic, multi-source side-channel attack that is orientation-aware, robust to execution-mode throttling, and transparent about leakage mechanisms, while quantifying low-cost mitigations with accuracy–overhead trade-offs.

In this paper, we present *AirSpy* (overview in Fig. 1), a practical black-box keystroke-inference attack that operates with *app-level permissions only* (no screen/audio capture and no privileged APIs). AirSpy fuses three time-synchronized observables: (i) 3D hand-joint positions, (ii) unit-quaternion hand rotations (rotx/roty/rotz/rotyw) capturing fine-grained orientation dynamics, and (iii) lightweight per-frame *PAL counters*—performance/memory telemetry exposed by the runtime (e.g., draw calls and frame timing). Leveraging the complementarity between PAL (event anchoring) and XR kinematics (discrimination), AirSpy first segments candidate keypress windows and then performs attribution-guided feature selection followed by a compact Transformer–DNN decoder for key inference. This design remains effective in both Foreground (FG) and Background (BG) execution without external sensors or privileged

hooks. Nevertheless, implementing AirSpy raises several key challenges:

1) Low SNR and BG throttling (blurred boundaries). Under Background (BG) policies, both XR inputs and PAL counters are *downsampled*, callbacks are *coalesced/jittered*, and XR/PAL clocks can drift, so onset/offset cues in any *single* stream become unreliable; naïve peak-picking either misses first/short keystrokes or merges adjacent ones.

Our Solution: we detect candidate windows using PAL *morphology* around inputs—an initial keypress produces a short burst in Draw Calls, follow-ups show smaller but consistent fluctuations, and Enter induces a brief Mesh-Faces dip with rapid recovery—while jointly checking local quaternion dynamics to suppress false positives. We then denoise XR poses via moving-average + Kalman filtering, *time-align* XR and PAL on a common grid with median/MAD outlier suppression, and run a lightweight *Transformer-based* segmenter that leverages windowed attention rather than hard thresholds. The segmentation–alignment stage stabilizes boundaries in both Foreground (FG) and BG, preserves micro-dynamics needed by the classifier, and tolerates reduced sampling without per-app heuristics.

2) Modeling fine-grained rotations (beyond 6DoF positions). Six Degrees of Freedom (6DoF) positions under-capture the fast fingertip *orientation* changes that drive Airpinch; purely positional features often confuse spatially adjacent keys and are brittle to small layout shifts.

Our Solution: we adopt scalar-first *unit quaternions* with right-multiplication; for successive poses we use the relative rotation $\Delta q_t = q_{t+\Delta t} \otimes q_t^{-1}$ and its small-angle *angular-velocity* proxy $\omega_t \approx \frac{2}{\Delta t} \text{vec}(\Delta q_t)$ to encode high-frequency orientation dynamics without Euler gimbal lock. Features combine short-window statistics (level/slope/energy), jerk of ω_t , and cross-correlations with PAL bursts. Position features mainly separate *where* a key occurs; quaternion features separate *how* the pinch is oriented. Their complementary error modes justify early fusion and measurably improve discrimination under subtle hand postures.

3) Cross-device/runtime generalization and transparency. Device generations and engine versions shift signal scales and PAL semantics; opaque models impede the design of principled mitigations.

Our Solution: we standardize features in a calibration-free manner, train a transfer-friendly hybrid *Transformer–DNN* decoder with augmentations that mimic deployment (downsampling, frame-drop, small quaternion jitter), and apply *post-hoc attribution* via *Shapley Additive Explanations (SHAP)* [25] over a >5k feature pool to retain the 97 most informative features. Attribution pinpoints which quaternion dynamics and which PAL counters dominate decisions, and we use these maps to stage mitigation studies (API rate limiting, counter sanitization). The resulting model maintains accuracy across FG/BG and device/runtime changes while remaining *explainable*, enabling quantitative defense trade-off analyses rather than ad-hoc tuning.

Results at a glance. On Meta Quest 2/3 across diverse apps, AirSpy achieves 93.56% Top-1 accuracy on 29 key classes with 92.8% segmentation recall in FG (in-domain).

Under restricted access, XR-only (pose streams only) yields 88.2%, PAL-only 74.8%, and BG throttled (XR+PAL) 90.1%. Downsampling XR+PAL to 10Hz retains 86.5%. In noise tests (10% random frame drops and 1° quaternion jitter on XR streams), accuracy remains > 86%. For cross-device transfer, we obtain up to 86.34% (Quest2→Quest3) and 85.72% (Quest3→Quest2) Top-1; user-independent evaluation reaches 88.9%. Runtime overhead is modest: median end-to-end latency ~13ms in FG and ~11.2ms in BG, with Frames Per Second (FPS) reductions <3 and <2 respectively.

In summary, the contributions of this paper are:

- **Multi-source XR side channel.** We formulate a keystroke-inference channel that jointly models *quaternion pose dynamics* and *PAL* counters under *app-level* permissions, effective in both FG and BG.
- **Transparent, deployable pipeline.** We provide an end-to-end design—PAL-shaped segmentation, statistical feature engineering/selection, and Transformer–DNN temporal decoding—plus post-hoc attribution (e.g., SHAP [25]) that quantifies per-signal leakage and informs defenses.
- **Robustness and transfer.** We validate across 29 keys, 10 users, and two device generations, including in-domain, user-independent, and cross-device settings, with down-sampling and noise ablations.
- **Low-cost countermeasures and overhead trade-offs.** We evaluate low-cost countermeasures against AI-enabled side-channels—specifically API rate limiting (down-sampling/jitter) and *PAL* counter sanitization (coarse-graining/background restriction)—and quantify their impact on accuracy and UI responsiveness alongside on-device latency/FPS/CPU–GPU overheads.

II. THREAT MODEL

A. Preliminaries

1) AR/VR System Background: Fig. 2 depicts the software–hardware stack we target. From bottom to top: *Hardware* comprises Sensors (accelerometer, gyroscope, magnetometer, camera, microphone, speaker) and Processing Units (CPU/GPU/NPU together with the image signal processor and display/codec engines). Above hardware, the *Low-level SDK* exposes Native APIs. These include (a) cross-platform interfaces such as OpenXR (Khronos) and graphics APIs (e.g., Vulkan, Metal), and (b) vendor stacks, e.g., ARCore NDK (anchors, Camera2/AndroidImageReader) on Android, ARKit/RealityKit + AV-Foundation/Metal on Apple, and WMR OpenXR/Research Mode on Windows. The *High-level SDK* (Unity/Unreal XR SDK) provides device-agnostic AR/VR functionality—Raycast/DirectGrab/UI/Locomotion, Motion Controller Components, and Tracked Geometries/Session Configuration—together with **PAL counters** (per-frame performance telemetry) and **PAL memory/allocation metrics**. On top, *Development Tool* layers (e.g., the AR/VR Interaction Toolkit) support AR/VR applications for Environmental sensing and HCI (hand/eye/face/voice/motion).

Commercial headsets allow *concurrent* applications. In our tested configurations, many high-level interfaces—especially

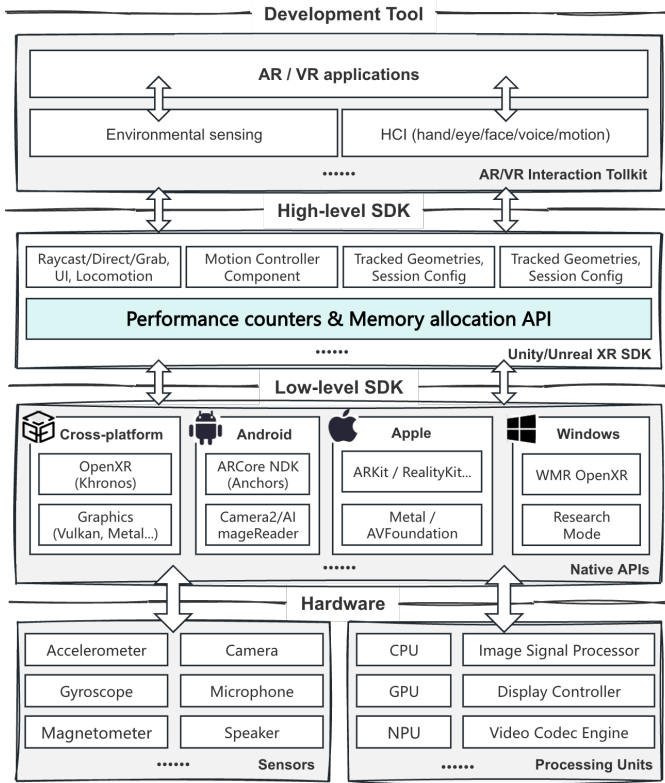


Fig. 2. General AR/VR software-hardware stack and the interfaces leveraged by AirSpy.

the performance/memory APIs—remain readable by background apps for responsiveness, whereas certain fine-grained input channels (e.g., full hand tracking) may prefer foreground focus by default. This design yields consistent software-layer observables across engines and platforms. We emphasize that our “BG” setting targets practical background policies where telemetry remains readable but is rate-limited (throttled), rather than a strict “frozen/paused” state in which the app process is fully suspended.

2) *Available Leakage Vectors*: AirSpy uses two *read-only*, *user-space* sources from Fig. 2:

(i) *PAL counters & Memory allocation (High-level SDK)*. Unity/Unreal provide per-frame statistics abstracted from device specifics. We group them as: *frame timing* (CPU/GPU frame time or FPS), *thread timing* (game/render/RHI), *rendering workload* (Number of draw calls, Vertex/Primitive count), and *process/graphics memory* (e.g., Unity AppMemoryUsage; Unreal RHI texture/RT pools). Although scoped to the caller, these indicators react to user interactions and cross-app contention (e.g., draw-call spikes when the keyboard UI updates; transient memory/mesh changes on mode switches), which we later exploit for event segmentation.

(ii) *OpenXR hand tracking (Low-level → High-level path)*. Through **OpenXR (Khronos)**, the runtime exposes 3D hand joints and bone quaternions (30–120 Hz when available). While underlying implementations differ (ARCore/ARKit/WMR), OpenXR unifies the semantics; we fuse these pose streams with the counters above.

Scope and portability. We rely solely on documented SDK

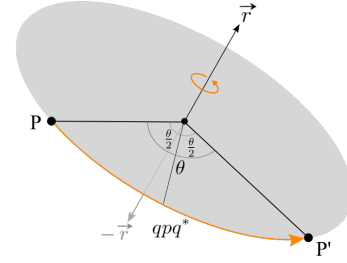


Fig. 3. Axis-angle interpretation of a 3D rotation: a point P is rotated around axis r by angle θ , represented by a unit quaternion q .

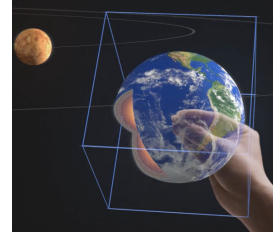


Fig. 4. Hand-tracking in AR/VR: quaternions encode the 3D orientations of joints, enabling accurate modeling of dynamic keystroke gestures.

interfaces at the levels highlighted in Fig. 2; no screen content, key-event hooks, privileged instrumentation, or extra sensors (e.g., Research Mode camera feeds) are used. For portability, we adopt a platform abstraction layer that maps the figure’s signals into four categories—**frame**, **thread**, **render**, and **memory**—and use them uniformly across devices/engines.

3) *Rotation Quaternions*: Rotation quaternions provide a compact and numerically stable representation of 3D orientations, avoiding the discontinuities of Euler angles and making them well-suited for real-time AR/VR signals. We use unit quaternions in scalar-first form $q = [w, x, y, z] = [w, \mathbf{v}]$, where $\mathbf{v} = [x, y, z]$ is the vector part.

The axis-angle interpretation of a 3D rotation is shown in Fig. 3. In AR/VR hand tracking, joint orientations are naturally represented by unit quaternions (see Fig. 4). A rotation around unit axis $\mathbf{u}$ by angle θ is written as

$$q = \left[\cos \frac{\theta}{2}, \mathbf{u} \sin \frac{\theta}{2} \right].$$

To capture motion dynamics, we compute relative rotation

$$\Delta q_t = q_{t+\Delta t} \otimes q_t^{-1}, \quad q^{-1} = [w, -x, -y, -z] \text{ for unit } q,$$

and derive an angular-velocity feature

$$\omega_t \approx \frac{2}{\Delta t} \text{vec}(\Delta q_t), \quad \text{vec}([w, x, y, z]) = [x, y, z],$$

which complements static orientations for keystroke inference.

B. Data Collection and Key Observations

To demonstrate the feasibility of attacks on current AR/VR systems, we developed a malware application to record both headset performance telemetry and XR hand-tracking. The application was built with Unity (v2020.3.26f1) and deployed on Meta Quest 2 and Quest 3 devices. It logs data at 60 Hz, including PAL metrics (CPU/GPU, Draw Calls, Mesh Stats, Memory Usage/Peak/Limit) as well as hand-tracking joint

positions and orientations. To improve robustness and cross-device generalization, we collected data from two devices with different sensors, involving 10 users of varying genders and heights. For each user, we recorded 26 letters and 3 symbols under both single-click and sequential inputs, totaling 55,000 *frame-level records* (one record per 60 Hz timestamp).

The system leverages the OpenXR API to collect three types of data:

- Hand position data: 3D coordinates (x, y, z) of hand skeletal joints
- Rotational quaternions: $(\text{rotx}, \text{roty}, \text{rotz}, \text{rotw})$ describing joint orientation
- System performance metrics: CPU/GPU usage, Draw Calls, Mesh Stats, Memory usage

OpenXR reports joint orientations as $(\text{rotx}, \text{roty}, \text{rotz}, \text{rotw})$ (vector part followed by scalar part). Throughout the paper, we convert it to the scalar-first convention $q = [w, x, y, z] = [\text{rotw}, \text{rotx}, \text{roty}, \text{rotz}]$ and normalize q to unit length when needed.

Considering user habits and device coverage, we selected a typical scenario: a user performing finger input using the Pinch mode in a VR environment. In this Air-pinch input model, the user maintains a considerable distance from the virtual keyboard, and the VR device uses raycasting to help the user accurately position their fingers. When the user pinches their fingers, the laser locks onto the key they intend to input, and releasing the fingers indicates the completion of an input. When the user opens the input interface, the virtual keyboard remains fixed and does not move with the device, aligning with common human input behavior.

Our key observation is that any interaction during VR input triggers dynamic changes in the underlying data. Specifically, when the user operates the virtual keyboard with their hands, various system parameters (such as PAL counters, rotation angles, etc.) exhibit characteristic fluctuations. Fig. 5 illustrates this process, and further analysis reveals that, during input, the number of Draw Calls typically experiences a significant spike. Each subsequent character input leads to smaller downward fluctuations, and when the Enter key is pressed, the fluctuation reaches a new peak, thereby completing the input process.

When the user types the first character, a significant increase appears in Draw Calls, with a rise of 10 drawing operations, followed by a return to lower levels within 3-5 frames. This change appears as a prominent peak in the time-series graph, and the second derivative of the performance metrics during the pinching action shows a positive peak value. The subsequent keystrokes show smaller variations, with an increase of 2 drawing operations, similarly returning to baseline within 3-5 frames.

C. PAL Leakage Mechanism

PAL counters are designed for performance profiling and runtime adaptation; therefore, they inevitably reflect fine-grained UI state updates. In air-pinch typing, each “hover/press/release” event triggers a short burst of UI rendering work (e.g., key highlight, text buffer update, cursor/candidate refresh), which changes the per-frame rendering

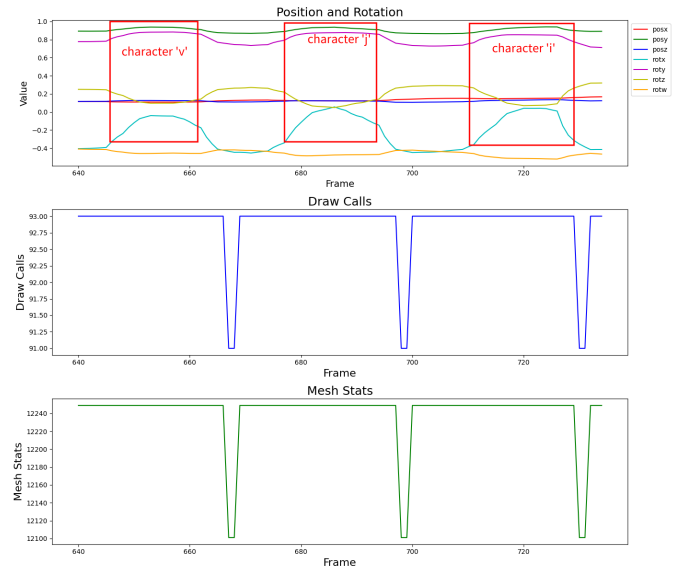


Fig. 5. Waveform patterns for keystroke detection: The first waveform change in the position family and rotation family represents the character ‘v’, the second represents the character ‘j’, and the third represents the character ‘i’.

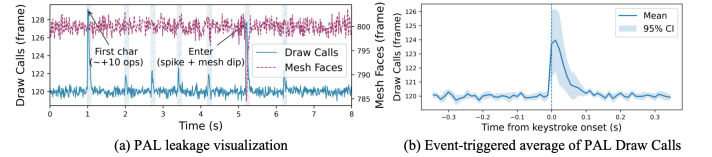


Fig. 6. Visualizing PAL feature leakage. (a) Representative PAL leakage visualization aligned to keystrokes. (b) Event-triggered average of PAL Draw Calls (mean $\pm$ 95% CI).

workload. As a result, PAL metrics such as Draw Calls/Mesh statistics exhibit transient spikes/dips tightly aligned with keystroke events, consistent with our observation that the first character produces a larger Draw Calls spike ($\approx +10$ ops) while subsequent characters yield smaller but repeatable fluctuations ($\approx +2\%$ ops), and Enter induces another peak. Fig. 6(a) shows a representative PAL trace where keystroke-aligned spikes/dips are clearly visible.

Importantly, these perturbations are short-lived (typically recovering within 3–5 frames) and repeat across inputs, making them reliable anchors for keystroke boundary detection even when XR streams are throttled or jittered in background mode. Fig. 6(b) further highlights the repeatable spike-and-recovery pattern around keystroke onset via an event-triggered average. This explains why PAL-only can still achieve high segmentation recall ($\approx 90\%$) despite losing pose semantics.

Since these PAL counters are exposed through user-space profiling interfaces (Table I), a non-privileged app can continuously sample per-frame rendering and memory statistics without external sensors, which turns performance telemetry into a side-channel for user input.

D. Threat Model and Assumptions

1) Adversary Capabilities: We assume the adversary controls a non-privileged AR/VR application installed on the target device. The adversary has no root privileges, cannot exploit kernel vulnerabilities, modify system binaries, or access external sensors such as microphones or cameras. Instead,

TABLE I
INTERFACES USED BY AIRSPY AND THEIR ACCESS BOUNDARY. FG: FOREGROUND; BG: BACKGROUND. RATES ARE THOSE OBSERVED IN OUR SETUP.
PAL COUNTERS ARE PER-FRAME STATISTICS (E.G., DRAW CALLS AND MESH FACES IN COUNT/FRAME).

Channel	Example APIs (engine)	Primary role	Scope	Rate (obs.)	FG access	BG access
XR streams (pose/kinematics)						
OpenXR hand tracking	OpenXR joints/quaternions (Unity/Unreal)	semantic motion cues (Stage-2/3)	per-user stream	30–120 Hz	Yes	Config-dependent
PAL counters (per-frame telemetry)						
Frame timing	FrameTimingManager / Stat GPU	auxiliary cues (Stage-2)	per-process	per-frame (≈ 60 Hz)	Yes	Yes
Thread timing	Game/Render/RHI timings	auxiliary cues (Stage-2)	per-process	per-frame (≈ 60 Hz)	Yes	Yes
Rendering workload	Draw calls, Mesh faces, Vertices, Primitives	boundary anchors (Stage-1)	per-process	per-frame (≈ 60 Hz)	Yes	Yes
Process/graphics mem	AppMemoryUsage / RHI texture/RT pools	auxiliary context (Stage-2)	per-process	low-to-per-frame	Yes	Yes

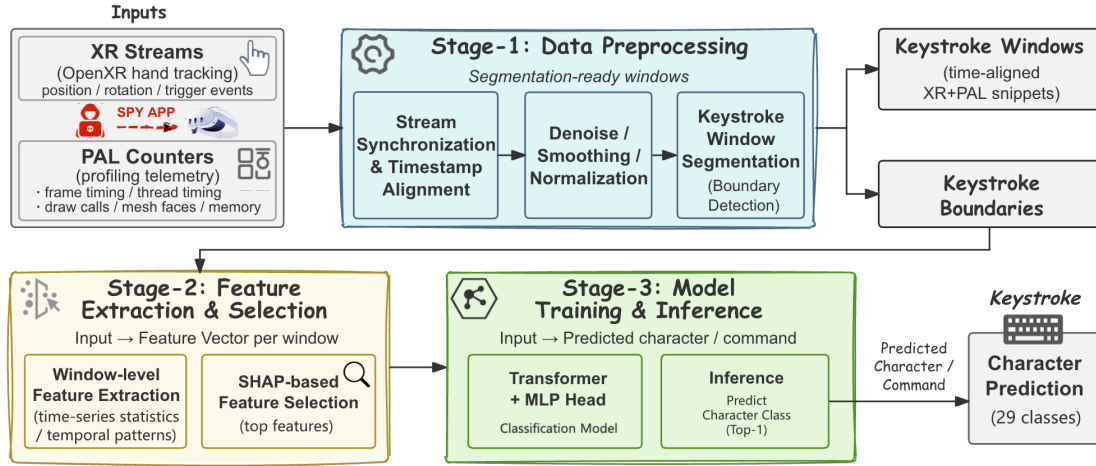


Fig. 7. Overview of AirSpy.

the attack relies solely on *user-space interfaces* exposed by standard XR runtimes and performance profiling APIs. These interfaces require only ordinary developer permissions and are widely used for debugging or runtime adaptation.

2) *Accessible Interfaces and Boundaries*: The attack leverages two families of legitimate APIs:

- **XR interfaces**: Hand-tracking APIs (e.g., OpenXR, Unity/Unreal XR SDKs) providing 3D joint positions and quaternions for gesture tracking.
- **PAL counters**: Per-frame rendering metrics (e.g., frame rate, draw calls, vertex counts) and memory statistics exposed by high-level SDKs.

Foreground applications can typically access these interfaces at full fidelity, while background applications may face reduced update rates or limited data availability depending on the platform. As summarized in Table I, the XR and PAL interfaces differ in scope, access rates, and availability across foreground (FG) and background (BG) modes.

3) *Access Modes*: To capture different platform restrictions, we consider four representative access modes:

- 1) **Full (FG)**: XR and PAL data at native rates with the app in the foreground.
- 2) **XR-only**: Only XR data available at native rates.
- 3) **PAL-only**: Only PAL data available at native rates.
- 4) **Throttled (BG)**: Both XR and PAL data accessible but downsampled to background-like rates.

These modes reflect realistic OS-level policies and allow us to assess how access constraints affect attack feasibility.

4) *Out-of-Scope Assumptions*: We explicitly exclude attacks requiring:

- Kernel or driver exploits;
- Screen scraping or framebuffer injection;
- External sensors beyond built-in XR runtimes (e.g., RGB cameras, IMUs outside the XR API scope).

Our focus is on the *realistic attack surface* exposed by standard user-space APIs under typical runtime conditions.

III. DESIGN OF AIRSPY

This section describes AirSpy's end-to-end pipeline (Fig. 7) for inferring VR virtual-keyboard inputs from two user-space telemetry families: XR hand pose streams and PAL PAL counters (Table I). We structure the design into three clearly separated stages with explicit I/O: **(Stage-1) Data Preprocessing** aligns multi-rate streams and segments continuous telemetry into per-keystroke windows; **(Stage-2) Feature extraction & selection** converts each window into time-series descriptors and uses SHAP to select the most informative features; **(Stage-3) Model training & inference** trains (i) a sequence segmentation model for keystroke boundaries and (ii) a Transformer–DNN classifier for key identification.

A. Stage-1: Data Preprocessing

AirSpy consumes three categories of telemetry streams, namely hand position coordinates, rotational quaternions, and system performance metrics that we refer to as PAL counters. The goal of Stage 1 is to convert asynchronous and noisy raw

TABLE II
THE MOST IMPORTANT FEATURES IDENTIFIED THROUGH SHAP ANALYSIS

Features	Definition
Rotx_aggre_linear_trend	Calculates a linear least-squares regression for aggregated values of the time series after applying a rotation around the x-axis
Rotz_chage_quantiles	First fixes a corridor defined by the quantiles ql and qh of the distribution of the time series after applying a rotation around the z-axis.
Y_minimum	Calculates the minimum value of the time series.
X_cwt_coefficients	Computes the Continuous Wavelet Transform (CWT) coefficients for the time series, capturing localized frequency information across different scales.
Rotw_fft_aggregated	Applies a Fast Fourier Transform to the time series after a rotation around the w-axis and aggregates the resulting frequency components.

streams into time aligned keystroke windows that are ready for feature extraction and learning. The pipeline includes the following steps.

- **Noise removal.** We combine moving average smoothing and Kalman filtering to suppress high frequency jitter in hand tracking signals and improve temporal stability for downstream inference.
- **Normalization.** All channels are normalized to zero mean and unit variance to ensure consistent scaling across heterogeneous sources, including XR kinematics and PAL counters.
- **Keystroke window segmentation.** We detect keystroke boundaries using transient and keystroke aligned perturbations in PAL signals, especially Draw Calls and Mesh Stats, which reflect workload changes during hover, press, and release actions.

Segmentation intuition and boundary patterns. Empirically, the first character often induces a prominent Draw Calls spike of about 10 operations that lasts for 3 to 5 frames, while subsequent characters yield smaller but repeatable fluctuations of about 2 operations. The Enter action typically causes a noticeable drop in Mesh Stats, for example about 90 faces, followed by rapid recovery, which serves as a termination marker. These short lived and repeatable PAL perturbations provide reliable anchors for detecting keystroke onsets and offsets in continuous streams.

Implementation note. To robustly locate boundaries under autocorrelation and temporal dependencies in performance telemetry, we use a Transformer-based sequence segmentation module (see §III-C). We run it on synchronized 60 Hz PAL+XR streams with a sliding window of $L=120$ frames (2.0s) and stride $S=10$ frames. For each window, the model outputs per-frame boundary scores $\hat{b}_t \in [0, 1]$, which we decode into keystroke boundaries via peak picking with non-maximum suppression. We define each keystroke window as the interval between two consecutive detected boundaries (start: onset boundary; end: the subsequent offset boundary). Boundary labels come from typing-logger timestamps aligned to the nearest 60 Hz frame, with a $\pm\delta = \pm 2$ -frame neighborhood marked positive. When computing segmentation recall, a predicted boundary is counted as correct if it falls within ± 2 frames (± 33 ms at 60 Hz) of the ground-truth boundary.

PAL-XR alignment and missing-point handling. We use PAL (native 60 Hz) as the reference timeline, and align XR hand-tracking samples to each PAL frame via nearest-neighbor timestamp matching (round-to-nearest). Let t_k be the timestamp of the k -th PAL frame and $\{(s_i, \mathbf{x}_i)\}$ be XR samples with timestamps s_i and features $\mathbf{x}_i$. For each t_k , we select

$$i^*(k) = \arg \min_i |s_i - t_k|.$$

If two XR samples are equidistant to t_k , we break ties by choosing the earlier one. If the matched XR sample is farther than a tolerance τ (we use $\tau = 1/120$ s) or is invalid (e.g., dropped tracking, NaNs), we treat it as missing and apply *synchronized dropping*—we remove the PAL frame and its XR match together so that the remaining PAL and XR sequences are strictly one-to-one aligned. We do not interpolate missing XR values to avoid introducing artificial smoothing inconsistent with true hand kinematics.

B. Stage-2: Feature Extraction and Selection

Given each segmented keystroke window, this stage converts raw multivariate time series into compact and informative descriptors for model learning. Our feature engineering pipeline follows a coarse to fine strategy.

- 1) **Initial feature extraction.** We extract more than 5,000 candidate statistical and temporal features from the windowed streams using time series analysis techniques such as distributional statistics, peak and energy descriptors, and temporal dynamics.
- 2) **Coarse grained feature embedding.** We leverage AutoGluon for dimensionality reduction to obtain a compact representation, which suppresses redundancy and improves robustness across conditions.
- 3) **Fine grained feature selection.** We apply SHAP to quantify feature contributions and identify the most influential features for prediction.

Role of SHAP. The SHAP analysis quantifies the marginal contribution of each feature to model outputs, yields a stable importance ranking, and provides interpretability of feature interactions and cross stream complementarity.

Through this process, we select the top $K=97$ features from the initial pool by ranking features with the mean absolute SHAP value computed on the validation set. We compute SHAP on the AutoGluon-selected tree-based model (gradient-boosted decision trees) using TreeSHAP. We determine K by a cumulative-contribution elbow rule: we choose the smallest K such that the normalized cumulative mean-|SHAP| of the top- K features reaches 95% of the total contribution, which yields $K=97$ in our dataset. Table II reports representative top features. We further validate the selected set using Pearson correlation heatmaps, principal component analysis, and random forest importance analysis to ensure that the final features are informative and non redundant.

C. Stage-3: Model Training and Inference

This stage infers the pressed character from each keystroke window. AirSpy adopts a hybrid Transformer and DNN architecture with two modules.

- **Transformer backbone.** A sequence encoder with self attention that captures temporal dependencies and contextual correlations within and across telemetry channels.
- **DNN head.** A lightweight fully connected classifier that maps the Transformer representation to the final character label.

Architecture details. The Transformer contains a data projection layer, positional encoding, stacked encoder blocks, and a pooling or aggregation layer such as mean pooling or CLS pooling. The DNN head uses multiple fully connected layers to output classification probabilities.

Label generation for VR keyboard inputs. To reduce manual labeling cost, we use a self-labeling procedure based on unsupervised clustering and a Hidden Markov Model (HMM). We first estimate the keyboard layout by clustering finger motion quaternions: because the relative pose between the head-mounted display and the keyboard is approximately fixed, repeated presses of the same key form stable clusters. We apply K-means with $K=38$ as an *over-clustering* step (to avoid merging nearby keys and to absorb non-key patterns such as idle/transition/Enter), and map clusters to key anchors by nearest-anchor assignment. We then refine pseudo-labels with an HMM on sequential typing, with emissions over the final 29 key classes (26 letters + space/comma/period). Enter is used only as a termination/segmentation cue and excluded from the 29-class accuracy; the resulting pseudo-labels are used for training without manual annotation.

Training strategy. We use a multi stage training procedure to improve accuracy and cross device generalization.

- **Pre training.** We pre train the Transformer on a large scale gesture dataset with more than 100,000 samples using a masked prediction objective in a BERT style to learn generalizable hand motion representations.
- **Transfer learning.** We initialize the keystroke inference model with the pre trained weights and fine tune progressively by gradually unfreezing deeper layers.
- **Parameter efficient adaptation.** We optionally use Low Rank Adaptation to reduce tuning cost while maintaining adaptability across devices and users.
- **Hyperparameter optimization.** We use Bayesian optimization to tune learning rate, attention heads, hidden dimensions, and dropout.

Optimization setup. We train with Adam using cosine learning rate scheduling and early stopping. In our setup, training converges after approximately 400 epochs on four NVIDIA V100 GPUs, balancing computation cost with robustness across conditions. Importantly, this training is an offline, one-time model development cost; the deployed attack only performs lightweight inference on the headset and incurs small runtime overhead as reported in our overhead evaluation.

IV. EXPERIMENTS

Our experimental design specifically addresses each of the challenges identified in the introduction. To validate our multi-source data fusion approach against the limitations of single-source methods, we conduct comparative experiments with various data source combinations. To evaluate our solution

TABLE III
XR/PAL ACCESS OBSERVED ON META QUEST DEVICES (2023-ERA;
NORMAL-APP PERMISSION ONLY; **Priv.: NORMAL**).

Dev.	OS	Runtime (tested)	Fields
Q2	Meta Quest OS v57.0 (Sep'23)	Unity 2022.3.0f1 OpenXR 1.8.2	XR: joints + quats (rotx/roty/rotz/rotyw). PAL: timing; draw calls/batches/mesh-face deltas; mem/alloc usage. Rate: PAL ≈ 60 Hz; XR 30–120 Hz (FG), BG throttled.
Q3	Meta Quest OS v57+ (Oct'23)	Unity 2022.3.0f1 OpenXR 1.8.2	Same fields as Q2.

for capturing complex Air-pinch gestures, we test our system against 29 different keystroke classes across two mainstream VR devices. To verify the effectiveness of rotational quaternions over traditional 6DoF approaches, we perform ablation studies comparing position-only, rotation-only, and combined feature sets.

This section presents comprehensive experimental results to validate the effectiveness of AirSpy. We conducted extensive experiments on two mainstream VR devices (Meta Quest 2 and Quest 3) with 10 volunteers of various genders and heights. The dataset includes 55,000 input samples covering 29 alphanumeric classes.

A. Experimental Setup

We collect data from 10 participants on two Meta Quest devices. We use a 7:2:1 train/val/test split for the Full (in-domain) setting; for LOUO and Device+User, splits are subject-disjoint (held-out subjects are excluded from train/val, and validation uses only training subjects). Models are trained on a server with four NVIDIA V100 GPUs using Adam for 400 epochs (batch size 8, learning rate 1×10^{-4}).

Metrics. We report Top- K accuracy ($K=1, 3, 5$), macro-averaged precision/recall/F1 across character classes, and confusion matrices for error analysis.

Statistical validation. We use 5-fold cross-validation and bootstrap confidence intervals. We further conduct leave-one-user-out evaluation to assess user-independent generalization.

Platform/runtime configuration and access boundary. To make the access claim reproducible, Table III reports the Quest OS / Unity / OpenXR versions under which XR and PAL streams were observed accessible, together with the retrieved fields, observed sampling rates, and FG/BG availability. All signals are collected by a *normal-permission* application (no screen/audio capture and no privileged APIs). We use “BG” to denote *throttled-but-not-frozen* execution; if the OS fully suspends the process, continuous sampling is not possible.

B. Results and Evaluation

1) *Comparative Model Performance:* Fig. 8 presents the classification results using various models and feature combinations. The Transformer-DNN model significantly outperforms traditional machine learning models in capturing temporal patterns and the correlations between high-dimensional data, especially in handling the complex hand pose variations introduced by rotational quaternion data.

TABLE IV
COMPARISON OF OUR METHOD WITH STATE-OF-THE-ART APPROACHES FOR VR/AR KEYSTROKE INFERENCE.

Method	Input Data Sources	Model Architecture	Permissions	Sampling Rate	Accuracy (%)	Number of Classes	External Devices	User Independence
VR-Spy [26]	WiFi CSI signals	Pattern Extraction	Normal App	2 kHz	92.50	12 (Gestures)	Yes (WiFi Router)	Moderate
Meteriz et al. [8]	Hand tracking traces	Multi-head CNN	Normal App	80 Hz	87.00	26 (Chars)	No	Moderate
HoloLogger [9]	Head motion (IMU)	CNN	Normal App	100 Hz	82.35	26 (Chars)	No	Moderate
Wu et al. [11]	Motion-Position Sensors	ResNet50 + LSTM	Normal App	60 Hz	85.92	30 (Keys)	No	Moderate
Zhang et al. [27]	PAL counters	ML (Random Forest)	Normal App	60 Hz	78.32	12 (Events)	No	Limited
Slocum et al. [20]	Head Motion (HMD)	CNN (Two-stage)	Normal App	60 Hz	82.00	5,000 (Words)	No	Moderate
Gopal et al. [12]	Video (2D Hand Joints)	CNN	High (Camera)	60 Hz	90.50	26 (Chars)	Yes	Strong
Heimdall [14]	Hand joints + Acoustic	RNN (GRU)	High (Microphone)	60 Hz	96.20	26 (Chars)	Yes	Strong
AirSpy (Ours)	Hand Pose + PAL	Transformer-DNN	Normal App	10–60 Hz	93.56	29 (Keys)	No	Strong

*Baseline note: We select baselines to represent XR/VR keystroke/text-input inference attacks and closely related interaction-inference systems. To make assumptions explicit, the table reports each method's access boundary via the *Permissions* and *External Devices* columns; AirSpy uses no screen/audio capture and no dedicated external sensors (normal-app only). Note: For Slocum et al. [20], accuracy reflects Top-5 word-level inference. VR-Spy [26] targets gesture recognition using WiFi signals. Our AirSpy demonstrates robust performance even under 10 Hz sampling rate during background throttling.

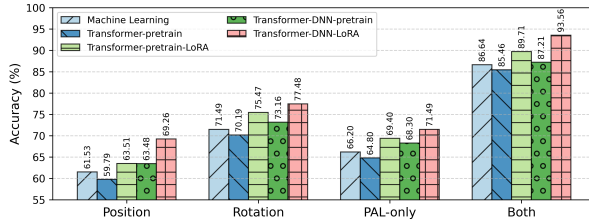


Fig. 8. Accuracy comparison of different models under four feature families (Position-only, Rotation-only, PAL-only, and All). For the final model (Transformer-DNN-LoRA), Top-1 accuracy is 69.26% (Position-only), 77.48% (Rotation-only), 71.49% (PAL-only), and 93.56% (All). All variants share the same Transformer-based AirSpy backbone; differences reflect training/adaptation (pretraining/LoRA) and the prediction head (ML vs. DNN).

2) *Comparison with State-of-the-Art Methods:* Table IV provides a multi-dimensional comparison between AirSpy and state-of-the-art VR/AR keystroke inference methods.

Threat Model and Stealthiness. As shown in Table IV, AirSpy operates under a more constrained threat model than high-accuracy baselines that require sensitive Camera/Microphone permissions (e.g., Gopal et al. [12], Heimdall [14]) and/or external support. By fusing app-visible PAL counters with hand-joint quaternions, AirSpy achieves superior stealthiness while maintaining competitive accuracy, outperforming prior performance-counter-based methods (e.g., Zhang et al. [27]) by 15.24%.

Accuracy and Inference Granularity. Despite the increased complexity of character-level inference over 29 classes, AirSpy demonstrates strong user independence and outperforms motion-based approaches such as Wu et al. [11] and Meteriz et al. [8]. While Slocum et al. [20] report 82.00% accuracy for word-level inference over a 5,000-word dictionary, AirSpy targets fine-grained individual keypresses, providing a more direct exploitation of typed input.

Robustness to System Constraints. Finally, Table IV highlights AirSpy's resilience to system-level background throttling. While vibration-sensitive or high-rate methods (e.g., HoloLogger [9]) benefit from dense sampling, AirSpy remains robust at a decimated 10 Hz sampling rate, matching practical OS constraints where many motion-dependent attacks degrade substantially.

3) *Cross-Device Generalization:* We also conducted cross-device experiments to evaluate the generalizability of different methods. Table V presents the results when models trained on Quest 2 data are evaluated on Quest 3 data, and vice versa.

TABLE V
CROSS-DEVICE GENERALIZATION ACCURACY.

Method	Quest 2 → Quest 3	Quest 3 → Quest 2
Zhang et al. [27]	70.21	68.47
Meteriz et al. [8]	76.53	73.92
Lee et al. [7]	74.89	72.15
Ours	86.34	85.72

The cross-device experiments reveal another significant advantage of our approach: better generalization across different devices. While existing methods experience a substantial accuracy drop (10-13 percentage points) when tested on a different device, our method maintains relatively high performance (7-8 percentage point drop), indicating greater robustness to device variations.

4) *Feature Importance Analysis:* To understand the contribution of different feature types to the model's performance, we conducted an ablation study by removing each feature family (position, rotation, and performance metrics) from the input. The results show that:

- Position features alone achieve 69.26% accuracy
- Rotation features alone achieve 77.48% accuracy
- Performance metrics alone achieve 71.49% accuracy
- Combined features achieve 93.56% accuracy

This analysis confirms that each feature type provides unique and complementary information for keystroke inference, with the combination of all features yielding the best performance.

5) *Error Analysis:* We conducted a detailed analysis of the confusion matrix to identify commonly confused character pairs and understand the underlying patterns of recognition errors. The top three most challenging character pairs are:

- 'b' and 'v' (confusion rate: 4.2%): These characters are adjacent on the standard QWERTY keyboard and require similar finger movements in the Air-pinch gesture
- 'm' and 'n' (confusion rate: 3.8%): These characters share similar rotational quaternion patterns during the pinch motion
- 'p' and 'o' (confusion rate: 3.5%): These characters have mirror-like positions on the keyboard, leading to similar hand trajectories

Our analysis reveals that most confusion occurs between keys that are physically adjacent on the virtual keyboard. This pattern suggests that the spatial proximity of keys creates overlapping feature distributions in the hand movement data. We

TABLE VI
USER AND DEVICE GENERALIZATION (95% CIs).

Setting	Segmentation recall (%)	Top-1 accuracy (%)
Full (in-domain, FG)	92.8 ± 1.4	93.56 ± 1.2
LOUO (unseen user)	90.5 ± 1.6	88.9 ± 1.5
Quest2 → Quest3 (unseen users)	88.7 ± 1.7	84.2 ± 1.6
Quest3 → Quest2 (unseen users)	89.1 ± 1.7	85.1 ± 1.5

TABLE VII
FALLBACK UNDER CONSTRAINED ACCESS. MODELS ARE FIXED; ONLY FEATURE AVAILABILITY/RATES VARY. REPORTED WITH 95% CIs.

Access mode	XR features	PAL features	Segmentation recall (%)	Top-1 accuracy (%)
Full (FG, native rates)	✓	✓	92.8 ± 1.4	93.56 ± 1.2
XR-only	✓		84.5 ± 1.8	88.2 ± 1.6
PAL-only		✓	90.1 ± 1.6	74.8 ± 2.1
XR+PAL (BG throttled)	✓	✓	87.3 ± 1.7	90.1 ± 1.5

also observed that characters with visually similar shapes (like 'p' and 'q') tend to have higher confusion rates, likely because users approach these keys with similar hand orientations.

Interestingly, the error patterns differ between position-based features and rotation-based features. While position features primarily confused adjacent keys, rotation features showed more confusion between keys that require similar finger orientations regardless of their positions. This finding highlights the complementary nature of these feature families and explains why their combination yields superior performance.

C. User and Device Generalization

a) *Protocol*: We evaluate (i) **LOUO**: train on all but one subject and test on the held-out subject, and (ii) **Device+User**: train on Quest 2 users only and test on unseen Quest 3 users (and vice versa). Models and hyper-parameters are kept fixed.

b) *Results*: Table VI shows that AirSpy maintains high accuracy under LOUO (Top-1 88.9%; recall 90.5%), and remains effective in the harder cross-device+new-user setting (Top-1 84.2–85.1%).

D. Impact of access constraints

Table VII evaluates AirSpy under restricted access regimes. With full access in the foreground, AirSpy achieves 92.8% segmentation recall and 93.56% Top-1 accuracy. When limited to **PAL-only**, segmentation remains relatively high at 90.1%, but Top-1 accuracy drops sharply to 74.8% (−18.8 pp), as pose cues from XR kinematics are lost. Conversely, **XR-only** retains moderate accuracy (88.2%) but suffers noisier onsets, reducing segmentation recall to 84.5% (−8.3 pp). Under **XR+PAL (BG throttled)**, both families are available but downsampled, yielding 87.3% recall and 90.1% accuracy—only −5.5 pp and −3.5 pp below full access, showing graceful degradation under realistic background policies. If a platform fully suspends background apps so that XR and PAL streams are not continuously accessible, AirSpy cannot run continuously. We treat this hard freezing as a strong OS level mitigation and therefore evaluate the more common throttling style background behavior.

TABLE VIII
RATE ROBUSTNESS UNDER BACKGROUND-LIKE SAMPLING. XR IS DOWNSAMPLED TO {60,30,10} HZ; PAL PER-FRAME IS DECIMATED TO {60,30,10} HZ ACCORDINGLY.

Configuration	Segmentation recall (%)			Top-1 accuracy (%)		
	60 Hz	30 Hz	10 Hz	60 Hz	30 Hz	10 Hz
XR-only	86.0	82.4	76.3	89.1	86.7	82.0
PAL-only	91.2	88.5	83.7	75.4	72.9	68.1
XR+PAL (throttled)	90.3	88.1	85.0	91.2	89.3	86.5
XR+PAL (FG, native rates)	—			93.56		

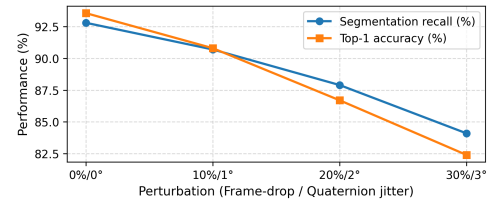


Fig. 9. Noise robustness for XR+PAL (FG). XR frame-drop rate and quaternion jitter (std. dev. in degrees) are applied before feature extraction.

E. Rate robustness under downsampling

Table VIII further reports performance as XR and PAL streams are downsampled to {60,30,10} Hz. For **XR-only**, Top-1 accuracy declines smoothly from 89.1% at 60 Hz to 82.0% at 10 Hz, while segmentation recall drops from 86.0% to 76.3%. For **PAL-only**, recall remains relatively robust (91.2%→83.7%) but accuracy decreases more steeply (75.4%→68.1%), reflecting limited semantic cues in PAL counters. The combined **XR+PAL (throttled)** consistently outperforms single-family modes at all rates (e.g., 86.5% accuracy at 10 Hz), confirming that multi-stream evidence mitigates performance loss even under aggressive sampling constraints.

a) *Noise robustness*: To emulate sensing/runtime perturbations we inject (i) random frame drops and (ii) quaternion jitter on XR streams; PAL counters are downsampled to the same effective rate for alignment. Fig. 9 reports smooth degradation—AirSpy retains > 86% accuracy up to 10% frame drops and 1° jitter, indicating headroom against moderate noise.

Defense-oriented evaluation. To systematically evaluate mitigations, we treat three OS level defense knobs as controllable variables. The first is *feature availability* via access control, including XR only, PAL only, and XR+PAL in Table VII. The second is *sampling rate throttling* at 60, 30, and 10 Hz in Table VIII. The third is *runtime perturbation* on XR streams, including frame drops and quaternion jitter in Fig. 9. Restricting high fidelity XR access using PAL only primarily hurts semantic inference, where Top 1 drops by 18.8 pp, while segmentation remains relatively high. Removing PAL using XR only mainly degrades onset detection, where recall drops by 8.3 pp, as shown in Table VII. Throttling down to 10 Hz suppresses both segmentation and classification, and XR only and PAL only suffer the most, while XR+PAL remains the most resilient across rates in Table VIII. Finally, AirSpy retains more than 86% accuracy under moderate perturbations, including 10% frame drops and 1° jitter in Fig. 9. Stronger jitter beyond 3° reduces accuracy below 85% in Section V-B,

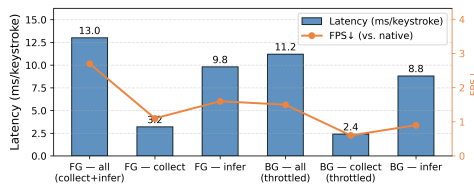


Fig. 10. Runtime overhead: latency per keystroke (bar) and FPS drop (line, right axis) across modes.

which indicates a practical security and usability trade off for runtime sanitization.

F. Latency and System Overhead

a) Protocol: We evaluate the runtime cost of AirSpy across four operational modes: *idle*, *collect* (XR+PAL streaming), *infer* (segmentation + classification), and *all* (collect + infer). Each mode runs under both *foreground* (FG) and *background* (BG) conditions.

We measure three key metrics:

- **Latency:** average per-keystroke end-to-end delay (ms),
- **FPS reduction:** frame-rate drop compared to the native VR app,
- **CPU/GPU overhead:** additional processing load beyond the base VR workload.

Each experiment is repeated 10 times to report stable averages.

b) Results: Fig. 10 and Fig. 11 summarize the runtime performance. In **foreground mode**, AirSpy incurs only **13 ms end-to-end latency** and less than **3 FPS reduction**, with **CPU/GPU overheads below 9% and 6%**, respectively.

In **background mode**, the platform's adaptive throttling further reduces overhead, with latency dropping to **11.2 ms** and frame-rate impact below **2 FPS**, demonstrating that our design remains lightweight even under continuous XR+PAL data streaming.

Overall, these results indicate that AirSpy imposes **minimal runtime costs** and can run alongside typical AR/VR applications without noticeably degrading the user experience.

G. Discussion and Limitations

AirSpy remains effective across diverse settings, but practical constraints and defenses merit discussion.

a) Attack practicality: AirSpy uses only user-space XR and PAL interfaces (no privileged access). Under realistic constraints, it remains effective: with moderate rate limits (e.g., 30 Hz) and small kinematic noise ($< 3^\circ$), accuracy stays above 86%, supporting feasibility in multi-app scenarios.

b) Limitations: AirSpy assumes throttled (not fully suspended) background access to XR/PAL telemetry in our evaluated settings; under full background suspension where such streams become unavailable, the attack cannot run continuously. (1) Stricter platform policies—e.g., restricting hand tracking to foreground apps or aggressively downsampling/sanitizing PAL counters—can significantly reduce accuracy (up to -18.8 pp in our experiments), so future OS/runtime updates may shrink the attack surface. (2) Although we observe cross-user/cross-device generalization, performance still

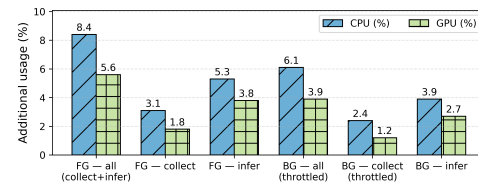


Fig. 11. Additional CPU/GPU usage under different modes.

lags in-domain training (e.g., Top-1 84.2–85.1%), motivating domain-adaptation and calibration-light techniques. (3) Our Transformer–DNN incurs nontrivial compute overhead (median ~ 13 ms latency; $< 9\%$ CPU), and designing lighter models is an important direction.

c) Implications for defense: Simple sanitization already helps: reducing the update rate to 10 Hz and/or injecting $> 3^\circ$ jitter noticeably degrades inference (e.g., XR-only drops to 82.0% Top-1 and overall accuracy falls below 85% under stronger jitter). More principled mitigations (e.g., adaptive privacy budgets or differential-privacy-style mechanisms for XR/PAL APIs) are promising directions to balance security and usability.

H. Countermeasures

Our empirical findings suggest practical defenses that reduce the attack surface while preserving benign XR functionality.

a) Throttling and sanitization at the API layer: Rate limiting and lightweight perturbations can directly disrupt temporal alignment and segmentation. As shown in Table VIII, throttling XR-only to 10 Hz reduces Top-1 accuracy from 93.56% to 82.0% and segmentation recall from 92.8% to 76.3%; adding random kinematic jitter above 3° further pushes accuracy below 85% (Fig. 9). These results motivate coarse-graining, quantization, rate throttling, and bounded noise injection for XR/PAL streams when full fidelity is unnecessary.

b) Access control and privilege separation: Limiting high-fidelity streams to foreground apps is a strong mitigation. In our constrained-access study (Table VII), restricting full-resolution hand tracking to foreground reduces Top-1 accuracy by 18.8 pp, and removing PAL signals (XR-only) reduces segmentation recall by 8.3 pp. This supports a tiered design: foreground apps may access full-resolution XR/PAL, while background apps receive downsampled/aggregated telemetry unless explicitly authorized by the user.

c) Policy enforcement and security–usability trade-offs: Beyond static policies, OS-level monitors can flag apps that poll XR/PAL APIs at anomalously high rates and throttle them dynamically. Overall, our evaluation quantifies three actionable defense knobs—access control (Table VII), rate throttling (Table VIII), and runtime perturbation (Fig. 9)—showing that lowering telemetry fidelity substantially reduces inference accuracy under moderate settings while retaining basic XR interaction and performance profiling utility.

V. RELATED WORK

This section reviews prior studies from three perspectives, namely side-channel attacks on VR and AR devices, inference

of virtual keystroke inputs, and deep-learning-based security attacks. Recent work has also revealed emerging immersive hijacking threats in VR systems. By summarizing the strengths and limitations of existing methods, we clarify the motivation and necessity of this study.

A. Side-Channel Attacks on VR/AR Devices

Research on side-channel attacks against VR and AR devices has uncovered multiple information leakage pathways.

Sensor data leakage. Ling et al. [1] were among the first to study side-channel attacks on HMD-based VR systems and showed that computer-vision-based and motion-sensor-based signals can infer user passwords entered via VR input devices. Casey et al. [2] pioneered immersive VR attacks and demonstrated that users can be physically manipulated through environment modifications. Zhang et al. [27] showed that PAL counters in VR devices can be exploited to infer user gestures and voice commands, exposing vulnerabilities in device telemetry. Shi et al. [28] leveraged speech-associated facial dynamics captured by motion sensors to infer speaker gender, identity, and speech content. These studies motivate the use of system performance metrics as attack features. Our work further combines such telemetry with hand movement data to build a more comprehensive inference model. Gopal et al. [12] proposed Hidden Reality, a video-based side-channel attack that decodes user inputs by analyzing hand gestures in immersive environments without direct observation of the virtual screen.

Spatial information leakage. Meyer-Lee et al. [15] and Farrukh et al. [16] demonstrated precise user location tracking by analyzing network traffic and 3D spatial map features.

Device state leakage. Yang et al. [29] inferred users' visual and audio activities by analyzing variations in the charging current of VR devices. Zhang et al. [30] leveraged sensor data to infer user gender and identity. These results highlight serious privacy risks in VR device signals, while most rely on a single data source. In contrast, our multi-source fusion approach aims to provide broader evidence and stronger inference capability.

B. Virtual Keystroke Input Inference Research

Keystroke inference is a core topic in VR and AR security. Existing studies mainly follow the approaches below.

Gesture tracking attacks. Sun et al. [8] reported 87% inference accuracy by analyzing hand motion data in an air-typing scenario. Yu et al. [10] reconstructed keyboard layouts by analyzing hand data from virtual avatars. Shang et al. [5] showed that network traffic patterns can reveal user locations in AR applications and highlighted an additional privacy risk. While these studies are closely related to our work, they primarily focus on conventional air-typing scenarios or speech recognition. Our work targets the more challenging air-pinch gesture.

External sensor attacks. Zhang et al. [13] used millimeter-wave radar reflections and point cloud analysis to identify inputs. Chen et al. [11] detected keystroke actions from changes in WiFi channel state. Luo et al. [14] proposed a placement-flexible acoustic attack that infers keystrokes from controller

clicking sounds using customized directional microphones and achieved over 96% accuracy. These methods require additional hardware.

Cross-modality attacks. Slocum et al. [20] showed that a malicious background app can infer hand-entered text using only head motion tracking, requiring zero permissions and achieving up to 82% top-5 word classification accuracy.

C. Application of Deep Learning in Security Attacks

Deep learning has become increasingly important in security attack research.

Temporal feature learning. Recent studies [31]–[34] indicate that Transformer architectures are effective for temporal modeling in attack scenarios. For example, GestFormer [31] achieved strong performance in gesture recognition, which motivates using Transformers for keystroke sequence analysis. Compared with traditional approaches, our Transformer–DNN hybrid better captures temporal dependencies during keystrokes.

Multimodal data fusion. Prior work [35], [36] has shown that deep models are effective for fusing multi-source signals. For example, [35] achieved promising results in multimodal fusion. Building on this line, our work introduces rotational quaternions into the attack model to improve the modeling of hand movements.

Explainability analysis. Explainability has attracted increasing attention in security research. Mosca et al. [21] provides instructive examples and recommendations. Our work applies SHAP to quantify feature importance and uses the attribution results to guide feature selection and improve model performance. Existing methods still face challenges in complex gesture input scenarios. The multimodal fusion design and deep learning framework in this paper aim to address these challenges and offer new insights for VR and AR security.

VI. CONCLUSION

In this paper, we propose AirSpy, a deployable, app-level keystroke side-channel on consumer VR headsets that exploits the *complementarity* of hand-pose kinematics and engine/runtime telemetry. By combining PAL-guided *morphological segmentation*, quaternion-aware orientation modeling, *SHAP*-driven feature selection (from $> 5k$ down to 97 features), and a compact *Transformer–DNN* decoder, AirSpy achieves 93.56% Top-1 across 29 keys with strong resilience to BG throttling, sensor noise, and cross-device/user shifts—while imposing *low runtime cost*. Our empirical study also evaluates *low-cost countermeasures*: reducing XR/PAL sampling rates or injecting small kinematic noise degrades accuracy in controllable ways, and *coarse-graining PAL counters* or restricting high-fidelity XR to foreground further narrows the attack surface with limited UI impact. These findings provide *practical guidance* for platform designers on balancing privacy and responsiveness in consumer XR. Finally, while we instantiate AirSpy on Meta Quest devices, similar OpenXR pose telemetry and PAL-style counters are likely to exist on other OpenXR-based platforms. The exact PAL fields and observable rates are platform/runtime-dependent and would need to be re-identified and validated when porting the attack.

REFERENCES

- [1] Z. Ling, Z. Li, C. Chen, J. Luo, W. Yu, and X. Fu, "I know what you enter on gear vr," in *2019 IEEE Conference on Communications and Network Security (CNS)*, 2019, pp. 241–249.
- [2] P. Casey, I. Baggili, and A. Yarramreddy, "Immersive virtual reality attacks and the human joystick," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 2, pp. 550–562, 2019.
- [3] Z. Yang, C. Y. Li, A. Bhalla, B. Y. Zhao, and H. Zheng, "Inception attacks: Immersive hijacking in virtual reality systems," *arXiv preprint arXiv:2403.05721*, 2024.
- [4] K. A. Awan, I. U. Din, A. Almogren, and J. J. P. C. Rodrigues, "Meditwin: A web 3.0-integrated digital twin for secure patient-centric healthcare in the metaverse," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 3, pp. 5654–5661, 2024.
- [5] J. Shang, S. Chen, J. Wu, and S. Yin, "Arspy: Breaking location-based multi-player augmented reality application for user location tracking," *IEEE Transactions on Mobile Computing*, vol. 21, no. 2, pp. 433–447, 2020.
- [6] S. J. Siddiqi, M. A. Jan, A. M. Basalamah, and M. Tariq, "Secure teleoperated vehicles in augmented reality of things: A multichain and digital twin approach," *IEEE Transactions on Consumer Electronics*, 2024.
- [7] J. Lee, "Tracking webvr user activities through hand motions: An attack perspective," *IEICE TRANSACTIONS on Information and Systems*, vol. 107, no. 8, pp. 1089–1092, 2024.
- [8] Ü. Meteriz-Yildiran, N. F. Yildiran, A. Awad, and D. Mohaisen, "A keylogging inference attack on air-tapping keyboards in virtual environments," in *2022 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. IEEE, 2022, pp. 765–774.
- [9] S. Luo, X. Hu, and Z. Yan, "Holologger: Keystroke inference on mixed reality head mounted displays," in *2022 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. IEEE, 2022, pp. 445–454.
- [10] Z. Yang, Z. Sarwar, I. Hwang, R. Bhaskar, B. Y. Zhao, and H. Zheng, "Can virtual reality protect users from keystroke inference attacks?" in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2725–2742.
- [11] Y. Wu, C. Shi, T. Zhang, P. Walker, J. Liu, N. Saxena, and Y. Chen, "Privacy leakage via unrestricted motion-position sensors in the age of virtual reality: A study of snooping typed input on virtual keyboards," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 3382–3398.
- [12] S. R. K. Gopal, D. Shukla, J. D. Wheelock, and N. Saxena, "Hidden reality: Caution, your hand gesture inputs in the immersive virtual world are visible to all!" in *32nd USENIX security symposium (USENIX Security 23)*, 2023, pp. 859–876.
- [13] L. Mei, R. Liu, Z. Yin, Q. Zhao, W. Jiang, S. Wang, S. Wang, K. Lu, and T. He, "mmspyvr: Exploiting mmwave radar for penetrating obstacles to uncover privacy vulnerability of virtual reality," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 8, no. 4, pp. 1–29, 2024.
- [14] S. Luo, A. Nguyen, H. Farooq, K. Sun, and Z. Yan, "Eavesdropping on controller acoustic emanation for keystroke inference attack in virtual reality," in *The Network and Distributed System Security Symposium (NDSS)*, vol. 2, 2024.
- [15] G. Meyer-Lee, J. Shang, and J. Wu, "Location-leaking through network traffic in mobile augmented reality applications," in *2018 IEEE 37th International Performance Computing and Communications Conference (IPCCC)*. IEEE, 2018, pp. 1–8.
- [16] H. Farrukh, R. Mohamed, A. Nare, A. Bianchi, and Z. B. Celik, "{LocIn}: Inferring semantic location from spatial maps in mixed reality," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 877–894.
- [17] G. M. Garrido, V. Nair, and D. Song, "Sok: Data privacy in virtual reality," *Proceedings on Privacy Enhancing Technologies*, vol. 1, pp. 21–40, 2024.
- [18] C. Slocum, Y. Zhang, E. Shayegani, P. Zaree, N. Abu-Ghazaleh, and J. Chen, "That doesn't go there: Attacks on shared state in {Multi-User} augmented reality applications," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2761–2778.
- [19] A. Giarretta, "Security and privacy in virtual reality: a literature survey," *Virtual Reality*, vol. 29, no. 1, p. 10, 2024.
- [20] C. Slocum, Y. Zhang, N. Abu-Ghazaleh, and J. Chen, "Going through the motions: {AR/VR} keylogging from user head motions," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 159–174.
- [21] E. Mosca, F. Szigeti, S. Tragianni, D. Gallagher, and G. Groh, "Shap-based explanation methods: a review for nlp interpretability," in *Proceedings of the 29th international conference on computational linguistics*, 2022, pp. 4593–4603.
- [22] R. Kumar, D. Javeed, A. Aljuhani, and A. K. M. N. Islam, "Blockchain-based authentication and explainable ai for securing consumer iot applications," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 1145–1154, 2024.
- [23] J. Pei, "F3: Fair federated learning framework with adaptive regularization," *Knowledge-Based Systems*, vol. 316, p. 113392, 2025.
- [24] L. Wang, X. Xu, and J. Pei, "Communication-efficient federated learning via dynamic sparsity: An adaptive pruning ratio based on weight importance," *IEEE Transactions on Cognitive Communications and Networking*, 2025.
- [25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.
- [26] A. Al Arafat, Z. Guo, and A. Awad, "Vr-spy: A side-channel attack on virtual key-logging in vr headsets," in *2021 IEEE Virtual Reality and 3D User Interfaces (VR)*. IEEE, 2021, pp. 564–572.
- [27] Y. Zhang, C. Slocum, J. Chen, and N. Abu-Ghazaleh, "It's all in your head (set): Side-channel attacks on {AR/VR} systems," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3979–3996.
- [28] C. Shi, X. Xu, T. Zhang, P. Walker, Y. Wu, J. Liu, N. Saxena, Y. Chen, and J. Yu, "Face-mic: inferring live speech and speaker identity via subtle facial dynamics captured by ar/vr motion sensors," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, 2021, pp. 478–490.
- [29] J. Li, Y. Meng, Y. Zhan, L. Zhang, and H. Zhu, "Dangers behind charging vr devices: Hidden side channel attacks via charging cables," *IEEE Transactions on Information Forensics and Security*, 2024.
- [30] T. Zhang, Z. Ye, A. T. Mahdad, M. M. R. R. Akanda, C. Shi, Y. Wang, N. Saxena, and Y. Chen, "Facereader: unobtrusively mining vital signs and vital sign embedded sensitive info via ar/vr motion sensors," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 446–459.
- [31] M. Garg, D. Ghosh, and P. M. Pradhan, "Gestformer: Multiscale wavelet pooling transformer network for dynamic hand gesture recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 2473–2483.
- [32] J. Pei, J. Li, Z. Song, M. M. Al Dabel, M. J. Alenazi, S. Zhang, and A. K. Bashir, "Neuro-vae-symbolic dynamic traffic management," *IEEE Transactions on Intelligent Transportation Systems*, 2025.
- [33] J. Pei, V. Frasca, A. Al-Dulaimi, W. Liu, T. H. Aldhyani, A. K. Bashir, and S. Mumtaz, "Distributed large models training optimization with real-time wireless channel feedback," *IEEE Journal on Selected Areas in Communications*, 2025.
- [34] J. Pei, W. Li, and S. Mumtaz, "From routine to reflection: pruning neural networks in communication-efficient federated learning," *IEEE Transactions on Artificial Intelligence*, 2024.
- [35] L. Xiang, Y. Chen, W. Chang, Y. Zhan, W. Lin, Q. Wang, and D. Shen, "Deep-learning-based multi-modal fusion for fast mr reconstruction," *IEEE Transactions on Biomedical Engineering*, vol. 66, no. 7, pp. 2105–2114, 2018.
- [36] J. Pei, M. Omar, M. M. Al Dabel, S. Mumtaz, and W. Liu, "Federated few-shot learning with intelligent transportation cross-regional adaptation," *IEEE Transactions on Intelligent Transportation Systems*, 2025.